

UNIT 3:

Transaction Processing & Concurrency Control

Transaction processing

- Transaction processing in a [Database Management System](#) (DBMS) ensures data integrity by treating a series of database operations as a single, indivisible unit, either fully completing or failing entirely, a concept known as the [ACID properties](#).
- Key to this are [atomicity](#), which ensures all operations succeed or none do, [consistency](#), which maintains data validity, [isolation](#), preventing concurrent transactions from interfering with each other, and [durability](#), which makes committed changes permanent.

- What is a Transaction?
- A transaction is a logical unit of work that involves one or more database operations, such as reading, inserting, updating, or deleting data.
- For example, transferring money between bank accounts involves multiple steps: debiting one account, crediting another, and writing both new balances.
- These operations are grouped into a single transaction to ensure data remains consistent, even if the system crashes during execution.
- A transaction (set of operations) may be stand-alone specified in a high level language like SQL submitted interactively, or may be embedded within a program.

- The ACID Properties
- To maintain data integrity, every transaction in a DBMS must adhere to the ACID properties:
- **Atomicity:**
 - Either all operations within a transaction complete successfully, or none of them do. There's no partially completed state.
- **Consistency:**
 - A transaction must bring the database from one valid state to another valid state, ensuring all integrity constraints are met.
- **Isolation:**
 - Concurrent transactions must not interfere with each other. The execution of one transaction should not affect the outcome of another running simultaneously.
- **Durability:**
 - Once a transaction is committed, its changes are permanent and will survive any subsequent system failures.

- How DBMS Supports Transaction Processing
- DBMS uses various techniques to manage transactions:
- **Logging and Recovery:**
- The DBMS maintains a log of all transaction operations. If a system failure occurs, this log is used to undo incomplete transactions or re apply committed ones, ensuring durability.
- **Concurrency Control:**
- Mechanisms like locking prevent multiple transactions from accessing and modifying the same data simultaneously, thus maintaining isolation and preventing data corruption.
- **Transaction Boundaries:**
- Transactions are explicitly defined within application programs by using begin transaction and end transaction statements to delineate the start and end of a logical unit of work.

Transaction Processing

- **Single-User System:**
 - At most one user at a time can use the system.
- **Multiuser System:**
 - Many users can access the system concurrently.
- **Concurrency**
 - **Interleaved processing:**
 - Concurrent execution of processes is interleaved in a single CPU
 - **Parallel processing:**
 - Processes are concurrently executed in multiple CPUs.

SIMPLE MODEL OF A DATABASE (for purposes of discussing transactions):

- **A database** is a collection of named data items
- **Granularity** of data - a field, a record , or a whole disk block (Concepts are independent of granularity)
- Basic operations are **read** and **write**
 - **read_item(X)**: Reads a database item named X into a program variable. To simplify our notation, we assume that the program variable is also named X.
 - **write_item(X)**: Writes the value of program variable X into the database item named X.

READ AND WRITE OPERATIONS:

- Basic unit of data transfer from the disk to the computer main memory is one block. In general, a data item (what is read or written) will be the field of some record in the database, although it may be a larger unit such as a record or even a whole block.
- `read_item(X)` command includes the following steps:
 - Find the address of the disk block that contains item X.
 - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 - Copy item X from the buffer to the program variable named X.

READ AND WRITE OPERATIONS (contd.):

- **write_item(X)** command includes the following steps:
 - Find the address of the disk block that contains item X.
 - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 - Copy item X from the program variable named X into its correct location in the buffer.
 - Store the updated block from the buffer back to disk (either immediately or at some later point in time).

Two sample transactions

- FIGURE : Two sample transactions:
 - (a) Transaction T1
 - (b) Transaction T2

(a) T_1

read_item (X);
 $X := X - N$;
write_item (X);
read_item (Y);
 $Y := Y + N$;
write_item (Y);

(b) T_2

read_item (X);
 $X := X + M$;
write_item (X);

2 Transaction and System Concepts (1)

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
 - For recovery purposes, the system needs to keep track of when the transaction starts, terminates, and commits or aborts.
- **Transaction states:**
 - Active state
 - Partially committed state
 - Committed state
 - Failed state
 - Terminated State

- Recovery manager keeps track of the following operations:
 - **begin_transaction**: This marks the beginning of transaction execution.
 - **read** or **write**: These specify read or write operations on the database items that are executed as part of a transaction.
 - **end_transaction**: This specifies that read and write transaction operations have ended and marks the end limit of transaction execution.
 - At this point it may be necessary to check whether the changes introduced by the transaction can be permanently applied to the database or whether the transaction has to be aborted because it violates concurrency control or for some other reason.

- Recovery manager keeps track of the following operations (cont):
 - **commit_transaction**: This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.
 - **rollback** (or **abort**): This signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone.

Transaction and System Concepts (4)

- Recovery techniques use the following operators:
 - **undo**: Similar to rollback except that it applies to a single operation rather than to a whole transaction.
 - **redo**: This specifies that certain *transaction operations* must be *redone* to ensure that all the operations of a committed transaction have been applied successfully to the database.

State transition diagram illustrating the states for transaction execution

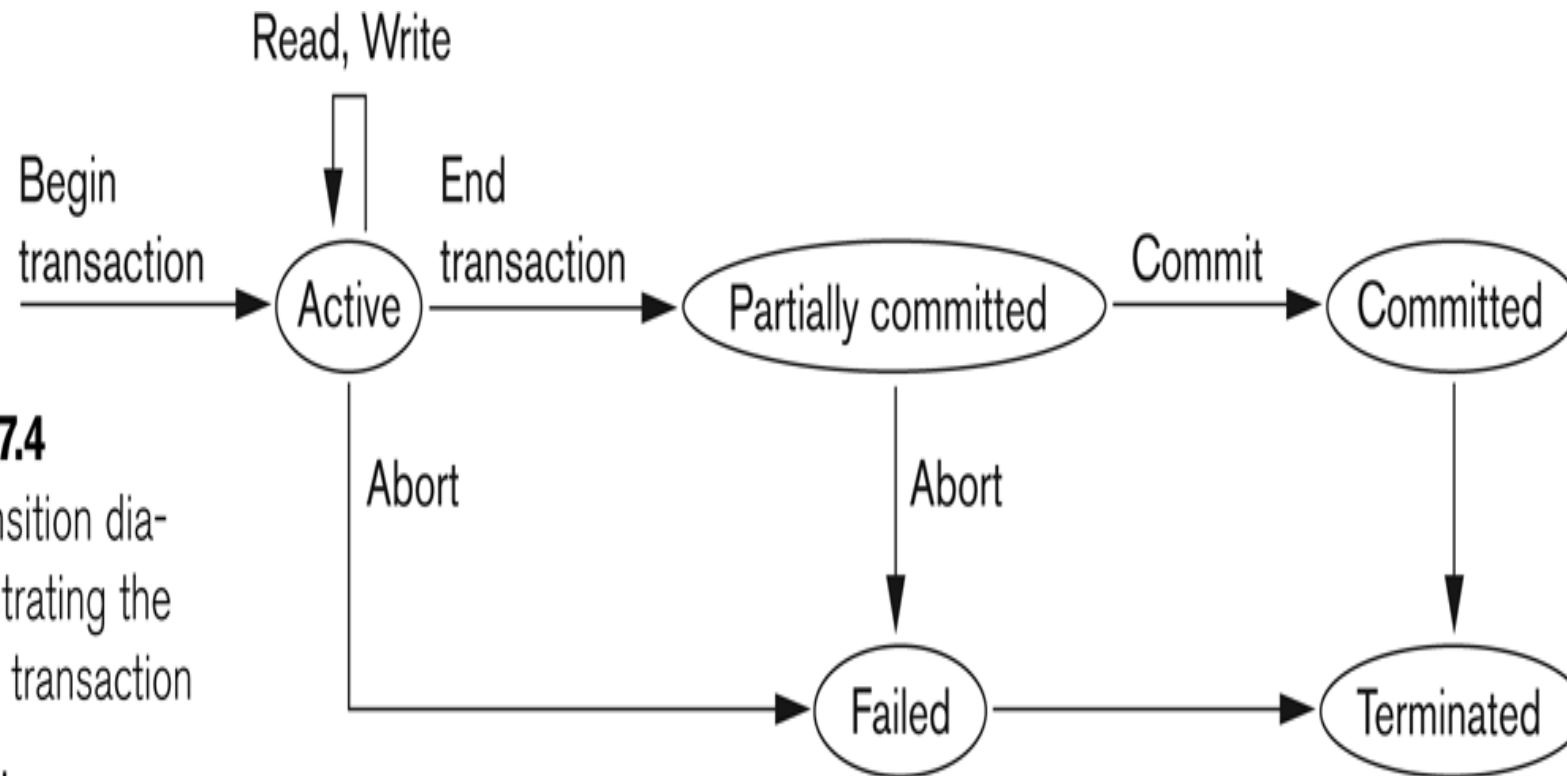


Figure 17.4

State transition diagram illustrating the states for transaction execution.

Why Concurrency Control is needed:

Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system. Advantages are:
 - **Increased processor and disk utilization**, leading to better transaction *throughput*
 - E.g., one transaction can be using the CPU while another is reading from or writing to the disk
 - **Reduced average response time** for transactions: short transactions need not wait behind long ones.
- **Concurrency control schemes** – mechanisms to achieve isolation
 - That is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database

- In a **multi-user system**, multiple users can access and use the same database at one time, which is known as the concurrent execution of the database. It means that the same database is executed simultaneously on a multi-user system by different users.
- While working on the database transactions, there occurs the requirement of using the database by multiple users for performing different operations, and in that case, concurrent execution of the database is performed.
- The thing is that the simultaneous execution that is performed should be done in an interleaved manner, and no operation should affect the other executing operations, thus maintaining the consistency of the database. Thus, on making the concurrent execution of the transaction operations, there occur several challenging problems that need to be solved.

Problems with Concurrent Execution

- In a database transaction, the two main operations are **READ** and **WRITE** operations. So, there is a need to manage these two operations in the concurrent execution of the transactions as if these operations are not performed in an interleaved manner, and the data may become inconsistent. So, the following problems occur with the Concurrent Execution of the operations:
- Lost Update Problems (W - W Conflict)
- Dirty Read Problems (W-R Conflict)
- Unrepeatable Read Problem (W-R Conflict)
- The Blind Write Problem
- Phantom Read Problem

Lost Update Problems (W - W Conflict)

- The problem occurs *when two different database transactions perform the read/write operations on the same database items in an interleaved manner (i.e., concurrent execution) that makes the values of the items incorrect hence making the database inconsistent.*
- **Consider the below diagram where two transactions T_x and T_y , are performed on the same account A where the balance of account A is \$300.**

Time	T_x	T_y
t_1	READ (A)	—
t_2	$A = A - 50$	
t_3	—	READ (A)
t_4	—	$A = A + 100$
t_5	—	—
t_6	WRITE (A)	—
t_7		WRITE (A)

- At time t_1 , transaction T_x reads the value of account A, i.e., \$300 (only read).
- At time t_2 , transaction T_x deducts \$50 from account A that becomes \$250 (only deducted and not updated/write).
- Alternately, at time t_3 , transaction T_y reads the value of account A that will be \$300 only because T_x didn't update the value yet.
- At time t_4 , transaction T_y adds \$100 to account A that becomes \$400 (only added but not updated/write).
- At time t_6 , transaction T_x writes the value of account A that will be updated as \$250 only, as T_y didn't update the value yet.
- Similarly, at time t_7 , transaction T_y writes the values of account A, so it will write as done at time t_4 that will be \$400. It means the value written by T_x is lost, i.e., \$250 is lost.

Hence data becomes incorrect, and database sets to inconsistent.

Dirty Read Problems (W-R Conflict)

- The dirty read problem occurs *when one transaction updates an item of the database, and somehow the transaction fails, and before the data gets rollback, the updated database item is accessed by another transaction.*
- **Consider two transactions T_x and T_y in the below diagram performing read/write operations on account A where the available balance in account A is \$300:**

Time	T_x	T_y
t_1	READ (A)	—
t_2	$A = A + 50$	—
t_3	WRITE (A)	—
t_4	—	READ (A)
t_5	SERVER DOWN ROLLBACK	—

- At time t_1 , transaction T_x reads the value of account A, i.e., \$300.
- At time t_2 , transaction T_x adds \$50 to account A that becomes \$350.
- At time t_3 , transaction T_x writes the updated value in account A, i.e., \$350.
- Then at time t_4 , transaction T_y reads account A that will be read as \$350.
- Then at time t_5 , transaction T_x rolls back due to server problem, and the value changes back to \$300 (as initially).
- But the value for account A remains \$350 for transaction T_y as committed, which is the dirty read and therefore known as the Dirty Read Problem.

Unrepeatable Read Problem (W-R Conflict)

- *Also known as Inconsistent Retrievals Problem that occurs when in a transaction, two different values are read for the same database item.*
- **Consider two transactions, T_x and T_y , performing the read/write operations on account A, having an available balance = \$300. The diagram is shown below:**

Time	T_x	T_y
t_1	READ (A)	—
t_2	—	READ (A)
t_3	—	$A = A + 100$
t_4	—	WRITE (A)
t_5	READ (A)	—

- At time t_1 , transaction T_x reads the value from account A, i.e., \$300.
- At time t_2 , transaction T_y reads the value from account A, i.e., \$300.
- At time t_3 , transaction T_y updates the value of account A by adding \$100 to the available balance, and then it becomes \$400.
- At time t_4 , transaction T_y writes the updated value, i.e., \$400.
- After that, at time t_5 , transaction T_x reads the available value of account A, and that will be read as \$400.
- It means that within the same transaction T_x , it reads two different values of account A, i.e., \$ 300 initially, and after updation made by transaction T_y , it reads \$400. It is an unrepeatable read and is therefore known as the Unrepeatable read problem.

The Blind Write Problem

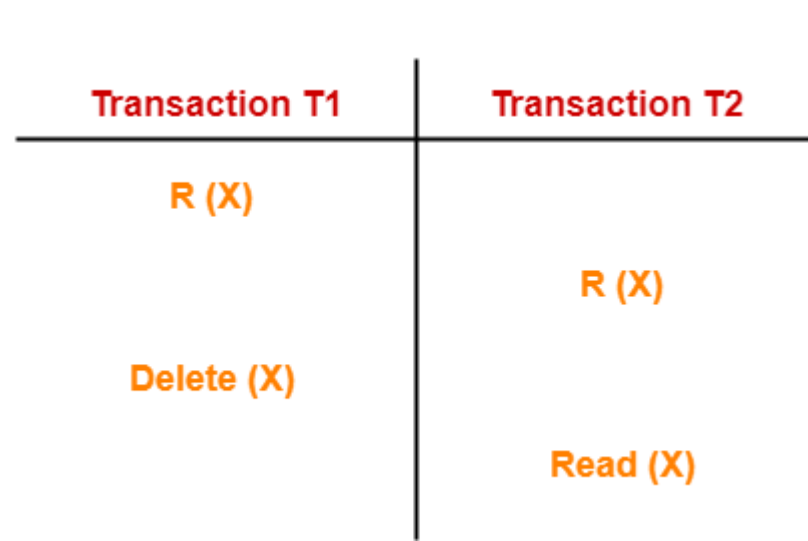
- If one transaction writes a value without reading it.

T1	T2	T3
	$R_2(X)$	
$R_1(X)$		
		$W_3(X)$
	$W_2(X)$	

- $W_3(X)$ is a blind write, as there is no read before write [$R_3(X)$ before $W_3(X)$]
- $W_2(X)$ is not a blind write, as a read happens before write [$R_2(X)$ before $W_2(X)$]

Phantom Read Problem-

- This problem occurs when a transaction reads some variable from the buffer and when it reads the same variable later, it finds that the variable does not exist.



Here,

- 1.T1 reads X.
- 2.T2 reads X.
- 3.T1 deletes X.
- 4.T2 tries reading X but does not find it.

In this example,

- T2 finds that there does not exist any variable X when it tries reading X again.
- T2 wonders who deleted the variable X because according to it, it is running in isolation.

- Thus, in order to maintain consistency in the database and avoid such problems that take place in concurrent execution, management is needed, and that is where the concept of Concurrency Control comes into role.

Why **recovery** is needed:

Transaction Processing

Why **recovery** is needed:

(What causes a Transaction to fail)

1. A computer failure (system crash):

A hardware or software error occurs in the computer system during transaction execution. If the hardware crashes, the contents of the computer's internal memory may be lost.

2. A transaction or system error:

Some operation in the transaction may cause it to fail, such as integer overflow or division by zero. Transaction failure may also occur because of erroneous parameter values or because of a logical programming error. In addition, the user may interrupt the transaction during its execution.

Transaction Processing

Why **recovery** is needed (Contd.):

(What causes a Transaction to fail)

3. Local errors or exception conditions detected by the transaction:

Certain conditions necessitate cancellation of the transaction. For example, data for the transaction may not be found. A condition, such as insufficient account balance in a banking database, may cause a transaction, such as a fund withdrawal from that account, to be canceled.

A programmed abort in the transaction causes it to fail.

4. Concurrency control enforcement:

The concurrency control method may decide to abort the transaction, to be restarted later, because it violates serializability or because several transactions are in a state of deadlock (see later).

Transaction Processing

Why **recovery** is needed (contd.):

(What causes a Transaction to fail)

5. Disk failure:

Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash. This may happen during a read or a write operation of the transaction.

6. Physical problems and catastrophes:

This refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, sabotage, overwriting disks or tapes by mistake, and mounting of a wrong tape by the operator.

Transaction and System Concepts

- **The System Log**

- **Log or Journal:** The log keeps track of all transaction operations that affect the values of database items.
 - This information may be needed to permit recovery from transaction failures.
 - The log is kept on disk, so it is not affected by any type of failure except for disk or catastrophic failure.
 - In addition, the log is periodically backed up to archival storage (tape) to guard against such catastrophic failures.

- The System Log (cont):
 - T in the following discussion refers to a unique **transaction-id** that is generated automatically by the system and is used to identify each transaction:
 - Types of log record:
 - [start_transaction,T]: Records that transaction T has started execution.
 - [write_item,T,X,old_value,new_value]: Records that transaction T has changed the value of database item X from old_value to new_value.
 - [read_item,T,X]: Records that transaction T has read the value of database item X.
 - [commit,T]: Records that transaction T has completed successfully, and affirms that its effect can be committed (recorded permanently) to the database.
 - [abort,T]: Records that transaction T has been aborted.

Commit Point of a Transaction:

- **Definition a Commit Point:**

- A transaction T reaches its **commit point** when all its operations that access the database have been executed successfully *and* the effect of all the transaction operations on the database has been recorded in the log.
- Beyond the commit point, the transaction is said to be committed, and its effect is assumed to be permanently recorded in the database.
- The transaction then writes an entry [commit, T] into the log.

- **Roll Back of transactions:**

- Needed for transactions that have a [start_transaction,T] entry into the log but no commit entry [commit, T] into the log.

Commit Point of a Transaction (cont):

- **Redoing transactions:**

- Transactions that have written their commit entry in the log must also have recorded all their write operations in the log; otherwise they would not be committed, so their effect on the database can be redone from the log entries. (Notice that the log file must be kept on disk.
- At the time of a system crash, only the log entries that have been written back to disk are considered in the recovery process because the contents of main memory may be lost.)

- **Force writing a log:**

- Before a transaction reaches its commit point, any portion of the log that has not been written to the disk yet must now be written to the disk.
- This process is called force-writing the log file before committing a transaction.

Schedules

- **Transaction schedule or history:**

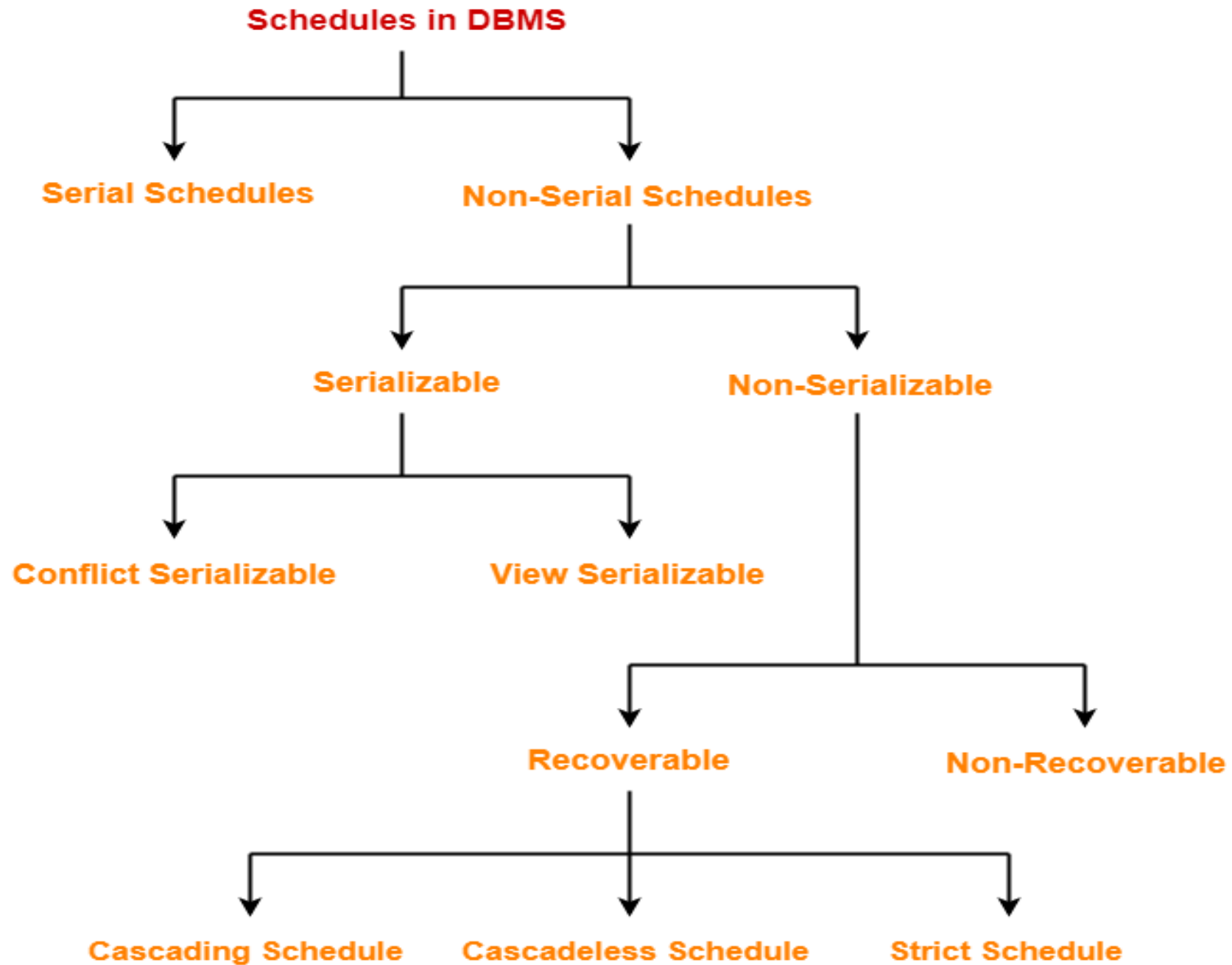
- When transactions are executing concurrently in an interleaved fashion, the order of execution of operations from the various transactions forms what is known as a transaction schedule (or history).

- A **schedule** (or **history**) S of n transactions $T_1, T_2, \dots, T_n$:

In DBMS transaction processing, a **schedule** (or **history**) is the actual, chronological order in which operations from multiple concurrent transactions are executed, either serially or with interleaved operations. Schedules are crucial for **concurrency control**.

Why are Schedules Important?

- Concurrency:** In a multi-user system, many transactions run simultaneously. A schedule determines the specific order in which their operations are interleaved.
- Data Consistency:** Without proper scheduling, interleaved operations can lead to data loss or inconsistencies.
- Transaction Integrity:** Schedules are fundamental to ensuring that transactions are processed reliably, even in the face of failures.



Transaction T1	Transaction T2
R (A)	
W (A)	
R (B)	
W (B)	
Commit	
	R (A)
	W (B)
	Commit

In this schedule,

- There are two transactions T1 and T2 executing serially one after the other.
- Transaction T1 executes first.
- After T1 completes its execution, transaction T2 executes.
- So, this schedule is an example of a **Serial Schedule**.

Non-Serial Schedules-

In non-serial schedules,

- Multiple transactions execute concurrently.
- Operations of all the transactions are interleaved or mixed with each other.

Transaction T1	Transaction T2
R (A)	
W (B)	
	R (A)
R (B)	
W (B)	
Commit	
	R (B)
	Commit

In this schedule,

- There are two transactions T1 and T2 executing concurrently.
- The operations of T1 and T2 are interleaved.
- So, this schedule is an example of a **Non-Serial Schedule**.

- **Finding Number Of Schedules-**

- Consider there are n number of transactions T1, T2, T3 , Tn with N1, N2, N3 , Nn number of operations respectively.

- **Total Number of Schedules-**

- Total number of possible schedules (serial + non-serial) is given by-

$$\frac{(N1 + N2 + N3 + \dots + Nn)!}{N1! \times N2! \times N3! \times \dots \times Nn!}$$

Total Number of Serial Schedules-

Total number of serial schedules

= Number of different ways of arranging n transactions

= n!

Total Number of Non-Serial Schedules-

Total number of non-serial schedules

= Total number of schedules – Total number of serial schedules

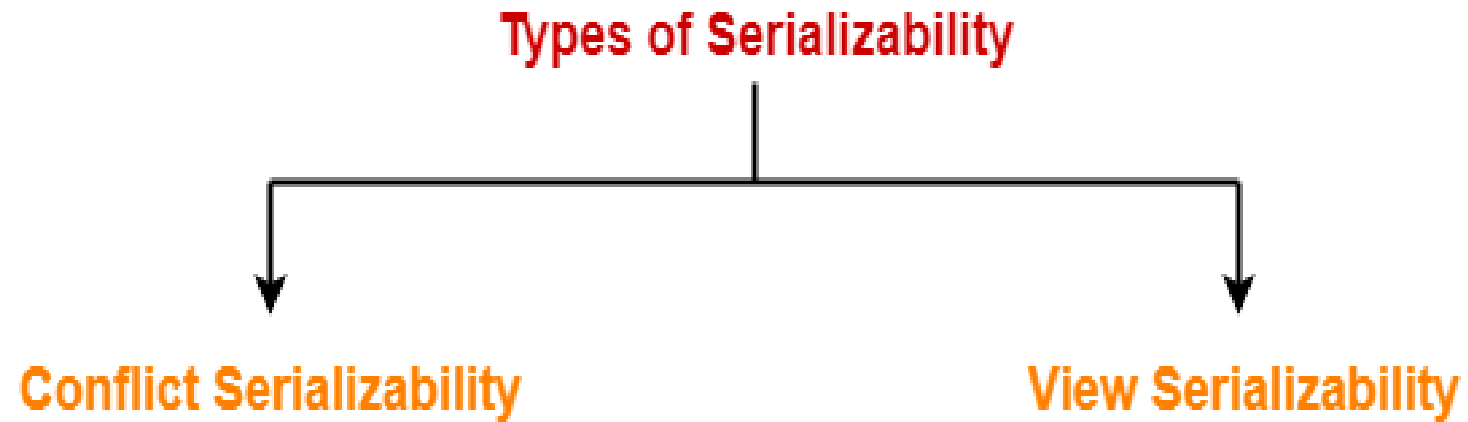
- Consider there are three transactions with 2, 3, 4 operations respectively, find-
- How many total number of schedules are possible?
- $(2+3+4)!/2! \times 3! \times 4! = 1260$
- How many total number of serial schedules are possible? $-3! = 6$
- How many total number of non-serial schedules are possible? -1254

- We have discussed-
- A schedule is the order in which the operations of multiple transactions appear for execution.
- Serial schedules are always consistent.
- Non-serial schedules are not always consistent.

Serializability in DBMS-

- Some non-serial schedules may lead to inconsistency of the database.
- Serializability is a concept that helps to identify which non-serial schedules are correct and will maintain the consistency of the database.
- **Serializable Schedules-**
- If a given non-serial schedule of 'n' transactions is equivalent to some serial schedule of 'n' transactions, then it is called as a **serializable schedule**.

Serializability is mainly of two types-



1. Conflict Serializability
2. View Serializability

Conflict Serializability-

If a given non-serial schedule can be converted into a serial schedule by swapping its non-conflicting operations, then it is called as a **conflict serializable schedule**.

Conflicting Operations-

Two operations are called as **conflicting operations** if all the following conditions hold true for them-

- Both the operations belong to different transactions
- Both the operations are on the same data item
- At least one of the two operations is a write operation

3 kinds of conflicts

RW (Read – Write) Conflict

WR (Write – Read) Conflict

WW (Write – Write) Conflict

Transaction T1	Transaction T2
Read(A)	
Write(A)	
	Read(A)
	Write(A)
Read(B)	
Write(B)	
	Read(B)
	Write(B)

In this schedule, Write(A)/Read(A) and Write(B)/Read(B) are called as **conflicting operations**. This is because all the above conditions hold true for them.

Transaction T1	Transaction T2
Read(A)	
Write(A)	
Read(B)	
Write(B)	
	Read(A)
	Write(A)
	Read(B)
	Write(B)

Thus, by swapping(non-conflicting) 2nd pair of the **read/write** operation of 'A' data item and 1st pair of the **read/write** operation of 'B' data item, this non-serial Schedule can be converted into a serializable Schedule. Therefore, *it is **conflict serializable***.

Consider the following schedule-

Transaction T1	Transaction T2
R1 (A)	
W1 (A)	
	R2 (A)
R1 (B)	

In this schedule,

- W1 (A) and R2 (A) are called as conflicting operations.
- This is because all the above conditions hold true for them.

Testing Conflict Serializability

Using Precedence Graph

- It is not always possible to test whether a given schedule is conflict serializable by manual methods because there may be thousands of transactions executing concurrently.
- Therefore, method which can test a given schedule for its conflict serializability is introduced using precedence graph.

Testing Conflict Serializability

Using Precedence Graph

- Looks at only read_Item (X) and write_Item (X) operations
- Constructs a precedence graph (serialization graph) - a graph with directed edges
- An edge is created from T_i to T_j if one of the operations in T_i appears before a conflicting operation in T_j
- The schedule is serializable if and only if the precedence graph has no cycles.

- 1] Check whether the given schedule S is conflict serializable or not-

- **S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$**

- **STEP 1:**

List all the conflicting operations and determine the dependency between the transactions-

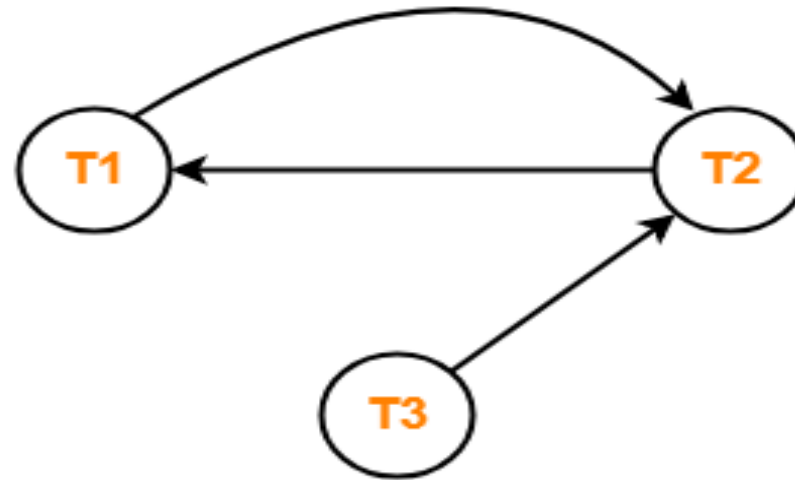
- $R_2(A)$, $W_1(A)$ ($T_2 \rightarrow T_1$)

- $R_1(B)$, $W_2(B)$ ($T_1 \rightarrow T_2$)

- $R_3(B)$, $W_2(B)$ ($T_3 \rightarrow T_2$)

Step-02:

Draw the precedence graph-



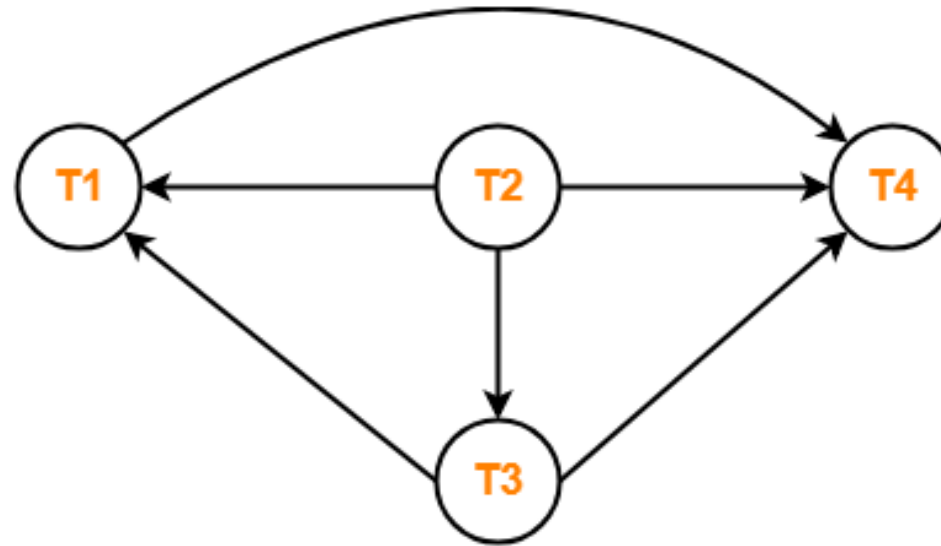
- Clearly, there exists a cycle in the precedence graph.
- Therefore, the given schedule S is not conflict serializable.

Check whether the given schedule S is conflict serializable and recoverable or not-

T1	T2	T3	T4
	R(X)		
		W(X) Commit	
W(X) Commit			
	W(Y) R(Z) Commit		
			R(X) R(Y) Commit

- List all the conflicting operations and determine the dependency between the transactions-
- $R_2(X)$, $W_3(X)$ ($T_2 \rightarrow T_3$)
- $R_2(X)$, $W_1(X)$ ($T_2 \rightarrow T_1$)
- $W_3(X)$, $W_1(X)$ ($T_3 \rightarrow T_1$)
- $W_3(X)$, $R_4(X)$ ($T_3 \rightarrow T_4$)
- $W_1(X)$, $R_4(X)$ ($T_1 \rightarrow T_4$)
- $W_2(Y)$, $R_4(Y)$ ($T_2 \rightarrow T_4$)

Draw the precedence graph-



- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S is conflict serializable.

Determine all the possible serialized schedules for the given schedule-

T1	T2
R(A) A = A-10	
	R(A) Temp = 0.2 x A W(A) R(B)
W(A) R(B) B = B+10 W(B)	

The given schedule S can be rewritten as-

T1	T2
R(A)	
	R(A)
	W(A)
	R(B)
W(A)	
R(B)	
W(B)	
	W(B)

Checking Whether S is Conflict Serializable Or Not-

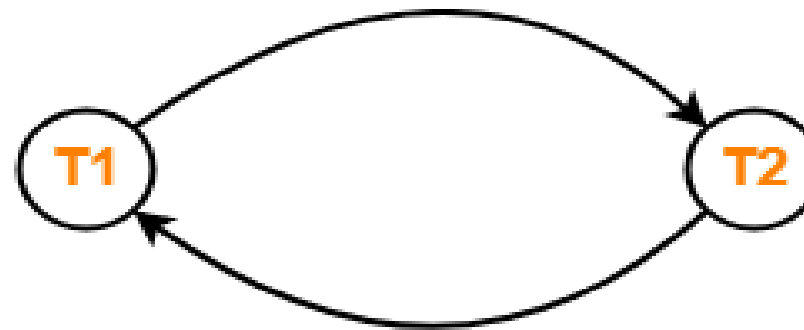
- Step-01:

List all the conflicting operations and determine the dependency between the transactions-

- $R_1(A) , W_2(A) (T_1 \rightarrow T_2)$
- $R_2(A) , W_1(A) (T_2 \rightarrow T_1)$
- $W_2(A) , W_1(A) (T_2 \rightarrow T_1)$
- $R_2(B) , W_1(B) (T_2 \rightarrow T_1)$
- $R_1(B) , W_2(B) (T_1 \rightarrow T_2)$
- $W_1(B) , W_2(B) (T_1 \rightarrow T_2)$

Step-02:

Draw the precedence graph-



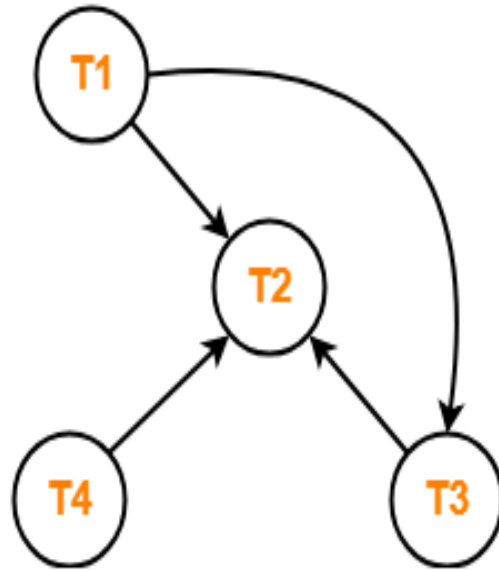
- Clearly, there exists a cycle in the precedence graph.
- Therefore, the given schedule S is not conflict serializable.
- Thus, Number of possible serialized schedules = 0.

- Check whether the given schedule S is conflict serializable or not. If yes, then determine all the possible serialized schedules-

T1	T2	T3	T4
			R(A)
	R(A)		
		R(A)	
W(B)			
	W(A)		
		R(B)	
	W(B)		

- List all the conflicting operations and determine the dependency between the transactions-
- $R_4(A)$, $W_2(A)$ ($T_4 \rightarrow T_2$)
- $R_3(A)$, $W_2(A)$ ($T_3 \rightarrow T_2$)
- $W_1(B)$, $R_3(B)$ ($T_1 \rightarrow T_3$)
- $W_1(B)$, $W_2(B)$ ($T_1 \rightarrow T_2$)
- $R_3(B)$, $W_2(B)$ ($T_3 \rightarrow T_2$)

Draw the precedence graph-



- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S is conflict serializable.

Finding the Serialized Schedules-

- All the possible topological orderings of the above precedence graph will be the possible serialized schedules.
- The topological orderings can be found by performing the [Topological Sort](#) of the above precedence graph.

After performing the topological sort, the possible serialized schedules are-

$T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2$

$T_1 \rightarrow T_4 \rightarrow T_3 \rightarrow T_2$

$T_4 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2$

View Serializability in DBMS

- In database systems, concurrent execution of transactions is used to improve resource utilization and system throughput. However, concurrency can lead to inconsistencies in the database if not handled properly.
- View Serializability ensures that even though transactions run concurrently, their outcome will be identical to the result achieved if the transactions were executed one by one in a specific order.
- It is a concept in concurrency control that checks whether a non-serial schedule produces the same final output as a serial schedule.
- It ensures data consistency by verifying that the initial reads, updated reads, and final writes of transactions are the same as in a serial schedule.

- **Methods to Check View Serializability**

- There are two methods to check View Serializability.

Method 1: View Equivalent

- If the given schedule is view-equivalent to the serial schedule itself then we can say that the schedule is view-serializable.
- To generate the serial schedule of the given schedule, we follow the following steps:
- Take the transaction say T_i that performs any operation first in the schedule. Write all the operations performed by that transaction one after the other in the given schedule.
- Similarly, take another transaction that is performing any operation right after T_i in the given schedule and write all the operations performed by it below transaction T_i on any data item.
- Follow the above steps for all the transactions. The order is very important.

View Serializability-

- If a given schedule is found to be view equivalent to some serial schedule, then it is called as a view serializable schedule.

- **View Equivalent Schedules-**

Consider two schedules S1 and S2 each consisting of two transactions T1 and T2.

- Schedules S1 and S2 are called **view equivalent** if the following three conditions hold true for them-
- 1. For each data item X, if transaction T_i reads X from the database initially in schedule S1, then in schedule S2 also, T_i must perform the initial read of X from the database.

Thumb Rule

“Initial readers must be same for all the data items”.

T1	T2	S1	S2	T1	T2
R(A)	W(A)			R(A)	W(A)

S1: T1 reads A from DB

S2: T1 reads A from T2

S1 and S2 are not satisfying first rule

- 2. If transaction T_i reads a data item that has been updated by the transaction T_j in schedule $S1$, then in schedule $S2$ also, transaction T_i must read the same data item that has been updated by the transaction T_j .

Thumb Rule

“Write-read sequence must be same.”.

T1	T2	T3	S1	S2	T1	T2	T3
W(A)					W(A)	W(A)	
	W(A)						
		R(A)					R(A)

S1: T3 reads A from T2

S2: T3 reads A from T1

S1 and S2 are not satisfying second rule

- 3. For each data item X, if X has been updated at last by transaction T_i in schedule S1, then in schedule S2 also, X must be updated at last by transaction T_i .

Thumb Rule

“Final writers must be same for all the data items”.

T1	T2	T3	S1	S2	T1	T2	T3
W(A)	R(A)	W(A)			W(A)	W(A)	W(A)
						R(A)	

S1: A is finally updated by T3

S2: A is finally updated by T1

S1 and S2 are not satisfying third rule

- A schedule is view serializable, if it is view equivalent to its serial schedule.
- Let us take the example of the table for checking **View Serializability**.
Let us call this **Schedule S1**

T1	T2
R(X)	
X=X+10	
W(X)	
	R(X)
	X=X+10
	W(X)
R(Y)	
Y=Y+20	
W(Y)	
	R(Y)
	Y=Y*1.1
	W(Y)

Also, for the above table the serial schedule is as follows:

Let us call this **Schedule S2**

As transaction T1 performed the first operation we wrote all of its operations on data item X as well as Y first and then all the operations of T2 are written next. Two schedules are said to be viewed as equivalent if they satisfy the following three conditions:

T1	T2	T1	T2
R(X)		R(X)	
X=X+10		X=X+10	
W(X)		W(X)	
	R(X)	R(Y)	
	X=X+10	Y=Y+20	
	W(X)	W(Y)	
R(Y)			R(X)
Y=Y+20			X=X+10
W(Y)			W(X)
	R(Y)		R(Y)
	Y=Y*1.1		Y=Y*1.1
	W(Y)		W(Y)

Condition 1: Initial Read

Initially if transaction T1 of Schedule S1 reads data item X, then transaction T1 of Schedule S2 must read data item X initially as well.

In the example above, the Initial read is made by the T1 transaction for both the data items X and Y in Schedule S1 and in S2 as well as initial read is made by transaction T1 for both. Thus, the initial read condition is satisfied in S1 and S2 for data items X and Y.

Condition 2: Update Read

If transaction Ti is reading updated data item X by transaction Tj in S1, then in Schedule S2 Ti should read X which is updated by Tj.

The value of X and Y is updated by transaction T1 in schedule S1, which is read by transaction T2. In Schedule S2 the X and Y data items are updated by T1 and are read by T2 as well. Thus, in the above two schedules, the update read condition is also satisfied.

Condition 3: Final Write

If transaction T1 updates data item Y at last in S1, then in S2 also, T1 should perform the final write operation on Y. The final write for data items X and Y are made by Transaction T2 in Schedule S1. Also, the final write is made by T2 in Schedule S2 in Schedule S1 as well as serial schedule S2. Thus, the condition for the final write is also satisfied.

Result

As all three conditions are satisfied for our schedule S1 and its Serial Schedule S2, they are view equivalent. Thus, we can say that the given schedule S1 is the view-serializable schedule and is consistent.

- **Method-02: Conflict Serializability and Dependency Graph**
- **Checking Whether a Schedule is View Serializable Or Not**

Step-01:

- Check whether the given schedule is conflict serializable or not.
- If the given schedule is conflict serializable, then it is surely view serializable. Stop and report your answer.
- If the given schedule is not conflict serializable, then it may or may not be view serializable. Go and check using other methods.

Thumb Rules

- All conflict serializable schedules are view serializable.
- All view serializable schedules may or may not be conflict serializable.

- **Step-02:**
- Check if there exists any blind write operation.
- (Writing without reading is called as a blind write).
- If there does not exist any blind write, then the schedule is surely not view serializable. Stop and report your answer.
- If there exists any blind write, then the schedule may or may not be view serializable. Go and check using other methods.

Thumb Rule

No blind write means not a view serializable schedule.

- **Step-03:**
- In this method, try finding a view equivalent serial schedule.
- By using the above three conditions, write all the dependencies.
- Then, draw a graph using those dependencies.
- If there exists no cycle in the graph, then the schedule is view serializable otherwise not.

- 1] Check whether the given schedule S is view serializable or not-

T1	T2	T3	T4
R (A)			
	R (A)		
		R (A)	
			R (A)
W (B)			
	W (B)		
		W (B)	
			W (B)

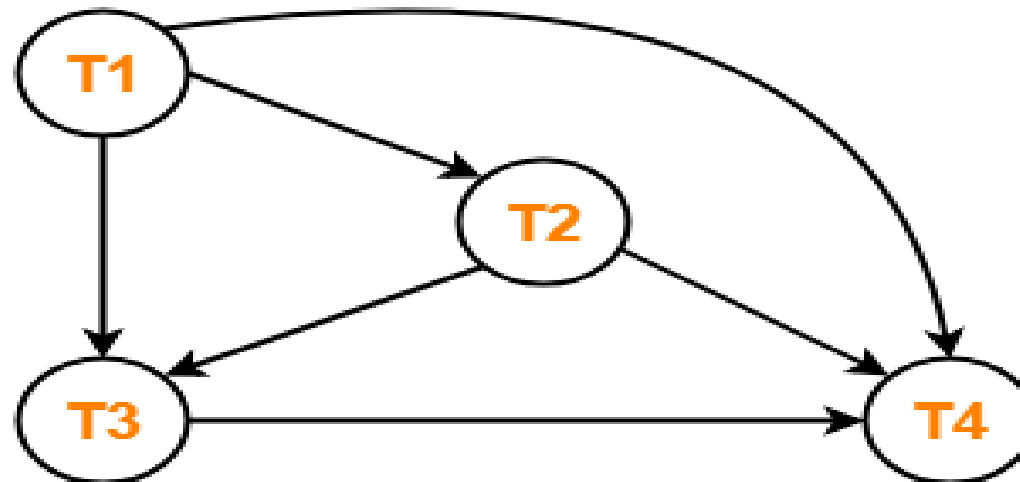
Solution-

- We know, if a schedule is conflict serializable, then it is surely view serializable.
- So, let us check whether the given schedule is conflict serializable or not.
- STEP 1:

List all the conflicting operations and determine the dependency between the transactions-

- $W_1(B)$, $W_2(B)$ ($T_1 \rightarrow T_2$)
- $W_1(B)$, $W_3(B)$ ($T_1 \rightarrow T_3$)
- $W_1(B)$, $W_4(B)$ ($T_1 \rightarrow T_4$)
- $W_2(B)$, $W_3(B)$ ($T_2 \rightarrow T_3$)
- $W_2(B)$, $W_4(B)$ ($T_2 \rightarrow T_4$)
- $W_3(B)$, $W_4(B)$ ($T_3 \rightarrow T_4$)

- **Step-02:**
- Draw the precedence graph-



- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S is conflict serializable.
- Thus, we conclude that the given schedule is also view serializable.

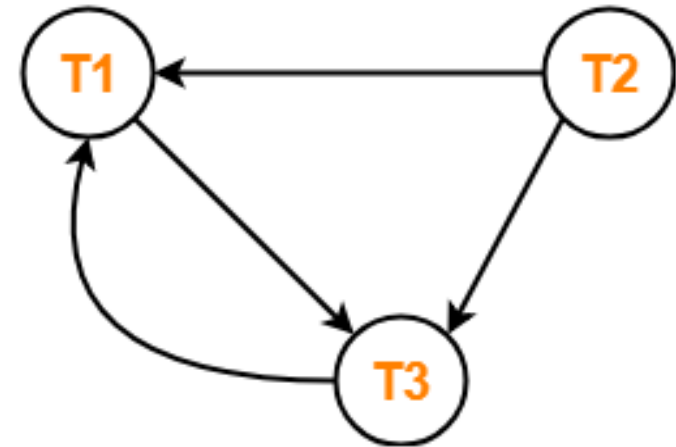
- 2] Check whether the given schedule S is view serializable or not-

T1	T2	T3
R (A)		
	R (A)	
		W (A)
W (A)		

- We know, if a schedule is conflict serializable, then it is surely view serializable.
- So, let us check whether the given schedule is conflict serializable or not.

- List all the conflicting operations and determine the dependency between the transactions-
- $R_1(A)$, $W_3(A)$ ($T_1 \rightarrow T_3$)
- $R_2(A)$, $W_3(A)$ ($T_2 \rightarrow T_3$)
- $R_2(A)$, $W_1(A)$ ($T_2 \rightarrow T_1$)
- $W_3(A)$, $W_1(A)$ ($T_3 \rightarrow T_1$)

Draw the precedence graph-



- Clearly, there exists a cycle in the precedence graph.
- Therefore, the given schedule S is not conflict serializable.

- Now, Since, the given schedule S is not conflict serializable, so, it may or may not be view serializable.
- To check whether S is view serializable or not, let us use another method.

Let us check for blind writes.

- **Checking for Blind Writes-**

- There exists a blind write $W_3(A)$ in the given schedule S.
- Therefore, the given schedule S may or may not be view serializable.

Now, To check whether S is view serializable or not, let us use another method.

- Let us derive the dependencies and then draw a dependency graph.

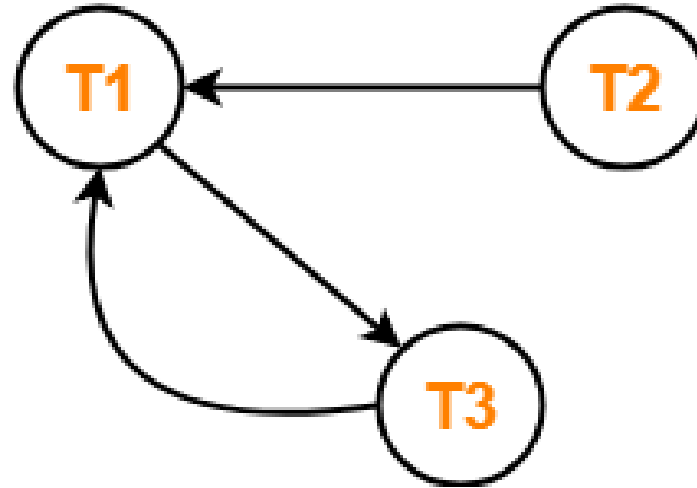
- Drawing a Dependency Graph-

- T1 firstly reads A and T3 firstly updates A.
- So, T1 must execute before T3.
- Thus, we get the dependency **T1 → T3**.
- Final updation on A is made by the transaction T1.
- So, T1 must execute after all other transactions.
- Thus, we get the dependency **(T2, T3) → T1**.
- There exists no write-read sequence.
- Now, let us draw a dependency graph using these dependencies-

T1	T2	T3
R (A)		
	R (A)	
		W (A)
W (A)		

$T1 \rightarrow T3.$

$(T2, T3) \rightarrow T1$



- Clearly, there exists a cycle in the dependency graph.
- Thus, we conclude that the given schedule S is not view serializable.

3] Check whether the given schedule S is view serializable or not-

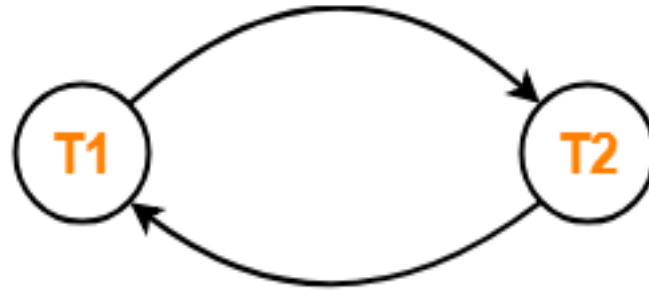
T1	T2
R (A)	
$A = A + 10$	
	R (A)
	$A = A + 10$
W (A)	
	W (A)
R (B)	
$B = B + 20$	
	R (B)
	$B = B \times 1.1$
W (B)	
	W (B)

- We know, if a schedule is conflict serializable, then it is surely view serializable.
- So, let us check whether the given schedule is conflict serializable or not.
- **Checking Whether S is Conflict Serializable Or Not-**
- **Step-01:**

List all the conflicting operations and determine the dependency between the transactions-

- $R_1(A) , W_2(A) (T_1 \rightarrow T_2)$
- $R_2(A) , W_1(A) (T_2 \rightarrow T_1)$
- $W_1(A) , W_2(A) (T_1 \rightarrow T_2)$
- $R_1(B) , W_2(B) (T_1 \rightarrow T_2)$
- $R_2(B) , W_1(B) (T_2 \rightarrow T_1)$

Draw the precedence graph-



- Clearly, there exists a cycle in the precedence graph.
- Therefore, the given schedule S is not conflict serializable.

Now,

- Since, the given schedule S is not conflict serializable, so, it may or may not be view serializable.
- To check whether S is view serializable or not, let us use another method.
- Let us check for blind writes.

- **Checking for Blind Writes-**

- There exists no blind write in the given schedule S.
- Therefore, it is surely not view serializable.

- **Alternatively,**

- You could directly declare that the given schedule S is not view serializable.
- This is because there exists no blind write in the schedule.
- You need not check for conflict serializability.

Practice

Let $r_i(z)$ and $w_i(z)$ denote read and write operations respectively on a data item z by a transaction T_i . Consider the following two schedules.

$S_1 : r_1(x) \ r_1(y) \ r_2(x) \ r_2(y) \ w_2(y) \ w_1(x)$

$S_2 : r_1(x) \ r_2(x) \ r_2(y) \ w_2(y) \ r_1(y) \ w_1(x)$

Which one of the following options is correct?

1. S_1 is conflict serializable, and S_2 is not conflict serializable.
2. S_1 is not conflict serializable, and S_2 is conflict serializable.
3. Both S_1 and S_2 are conflict serializable.
4. Neither S_1 nor S_2 is conflict serializable.

Conflict serializable or not

Time ↓

T1	T2	T3
Read(A) A := f ₁ (A)	Read(B)	
	B := f ₂ (B) Write(B)	Read(C)
Write(A)		C := f ₃ (C) Write(C)
	Read(A) A := f ₄ (A)	Read(B)
Read(C) C := f ₅ (C) Write(C)	Write(A)	
		B := f ₆ (B) Write(B)

Let $R_i(z)$ and $W_i(z)$ denote read and write operations on a data element z by a transaction T_i , respectively. Consider the schedule S with four transactions.

$$S : R_4(x)R_2(x)R_3(x)R_1(y)W_1(y)W_2(x)W_3(y)R_4(y)$$

Which one of the following serial schedules is conflict equivalent to S ?

- A. $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2$
- B. $T_1 \rightarrow T_4 \rightarrow T_3 \rightarrow T_2$
- C. $T_4 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2$
- D. $T_3 \rightarrow T_1 \rightarrow T_4 \rightarrow T_2$

Suppose a database system crashes once again while recovering from the previous crash. If we assume that checkpointing is not done by the database during the transactions or the recovery—Mark the correct statements.

- A) Undo and Redo list will be used to recover**
- B) Database becomes inconsistent**
- C) Any transactions that are already done or undone will not get recovered again.**
- D) The system will not recover further.**

ANSWER:- A

Explanation :Even after multiple crashes of the database system, the same undo and redo list will be used